# Code Explanation

**Loading Data to Target Table**

The provided code is a Python module named `loading.py` responsible for loading transformed data into a target database table using Apache Spark. It takes a SparkSession and a DataFrame as input, writes the data to the target table, and prints the number of records loaded.

**Overview of the Code**

This code defines a function `load_to_target` that loads data from a DataFrame into a target database table. It utilizes the SparkSession to write the data in JDBC format to the specified target table.

Step 1: Importing Necessary Modules and Loading Configuration
The code begins by importing the necessary modules, including `SparkSession` and `DataFrame` from `pyspark.sql`, and `Dict` from `typing`. It also imports configuration variables from a local module named `.config`.

```
from pyspark.sql import SparkSession, DataFrame
from typing import Dict
from .config import DB_CONFIG, TARGET_TABLE, BATCH_SIZE
```

Step 2: Defining the load_to_target Function
The `load_to_target` function is defined with two parameters: `spark` of type `SparkSession` and `data` of type `DataFrame`. This function is designed to load the provided DataFrame into a target database table.

```
def load_to_target(spark: SparkSession, data: DataFrame) -> None:
```

Step 3: Retrieving Target Database Configuration
Inside the `load_to_target` function, the code retrieves the target database configuration from the `DB_CONFIG` dictionary.

```
target_config = DB_CONFIG["target"]
```

Step 4: Writing Data to the Target Table
The code then writes the DataFrame to the target table using the `write` method of the DataFrame, specifying the JDBC format and various options such as the JDBC URL, database table name, username, password, driver, and batch size.

```
(data.write
 .format("jdbc")
 .option("url", target_config["jdbc_url"])
 .option("dbtable", TARGET_TABLE)
 .option("user", target_config["user"])
 .option("password", target_config["password"])
 .option("driver", target_config["driver"])
```

```
 .option("batchsize", BATCH_SIZE)
 .mode("append")
 .save())
```

Step 5: Printing the Number of Records Loaded
After successfully loading the data, the code prints a message indicating the number of records loaded into the target table.

```
print(f"Successfully loaded {data.count()} records to {TARGET_TABLE}
")
```